FIC – curs 1

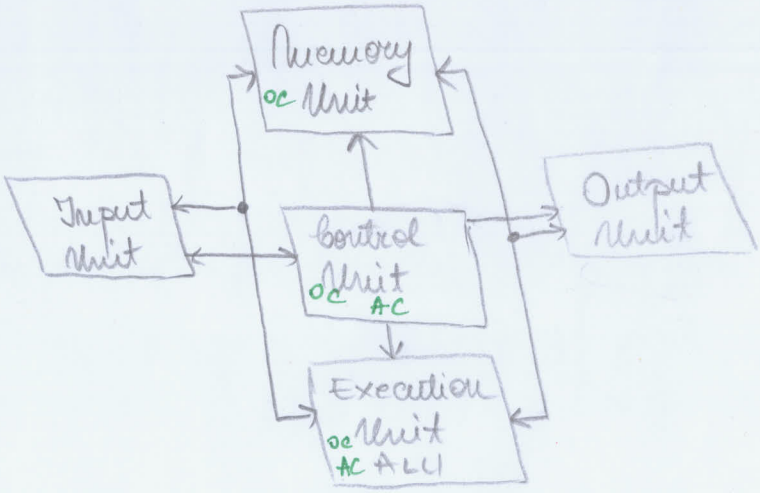
30.09.2016.

AC – Flavius
- reprezentarea datelor
  - numerice
  - non-numerice
- sumatoare
- înmulțitoare
- design automate

OC – Mihai
- înmulțitoare
- împărțitoare
- estimări de performanță
- memorii cache.

FIC – Lucian
- control
  - wired logic
  - microprogrammed logic
- IAS (mașina lui Von Neumann)
- set de instrucțiuni
- pipeline

* Nokia



ENIAC – Electronic Numerical Integrator and Calculator
1940 (5.06.1943 → University of Pennsylvania, Moore School of Electrical Engineering)

John von Neumann

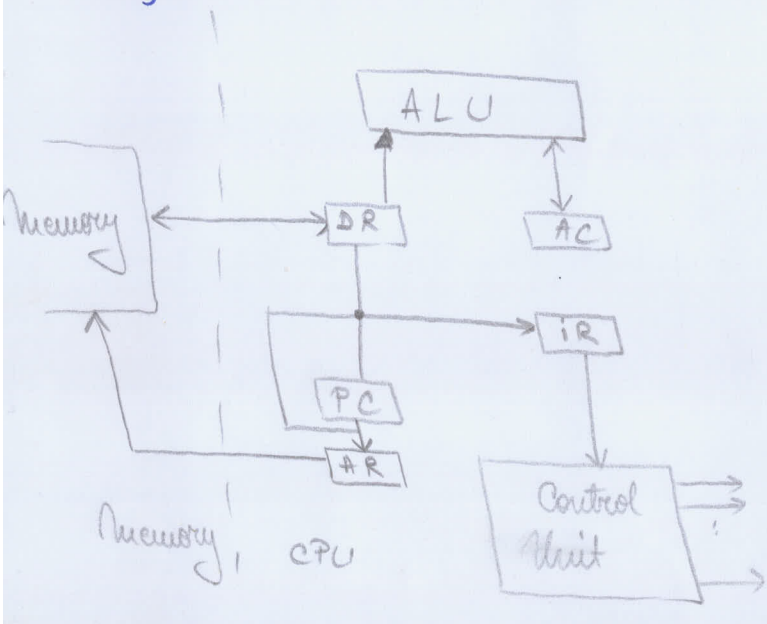
EDVAC – John. , P. Eckert.

I. Arhitecturi de bază ale unui calculator

• project characteristics

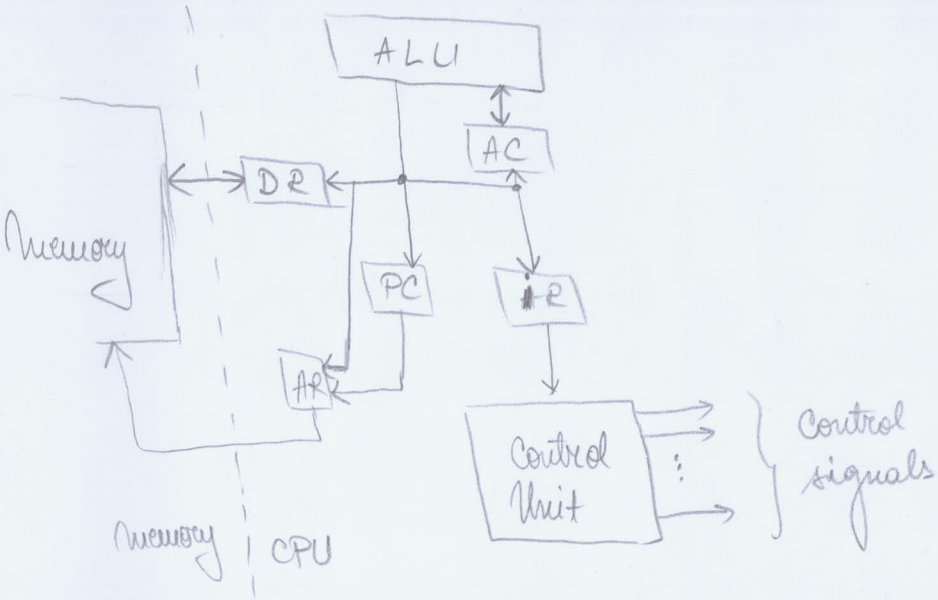
IAS - Institute for Advanced Study at Princeton University

Mașina IAS (var. 1).



DR - data register
AR - Address "
AC - Accumulator "
IR - Instruction "
PC - Program Counter

Mașina IAS (var. 2)



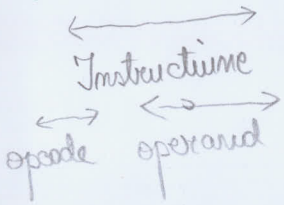
ENIAC → EDVAC → IAS
1943 1946 1945

IAS (von Neumann) → single memory for programmes and data

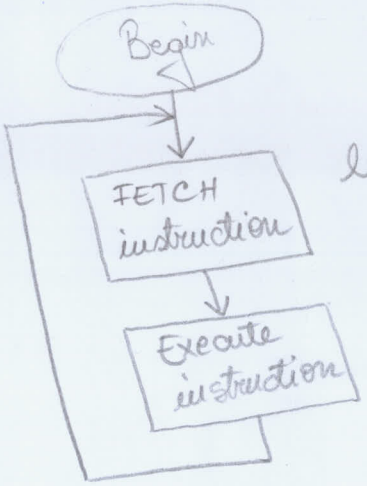
Harvard (microcontrollerele sunt de acest tip)

Instructiuni pe care le putem executa IAS:

- Load x → AC ← Mem[x]
- Store x → Mem[x] ← AC
- Add x → AC ← AC + Mem[x]
- And x → AC ← AC ∧ Mem[x]

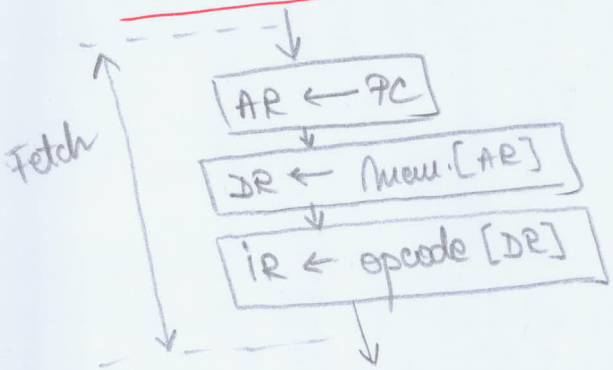


- Jump x → PC ← x
- JumpZ x → if AC = 0 then PC ← x
- COMP → AC ← $\overline{AC}$
- RightShift → AC ← RightShift (AC)

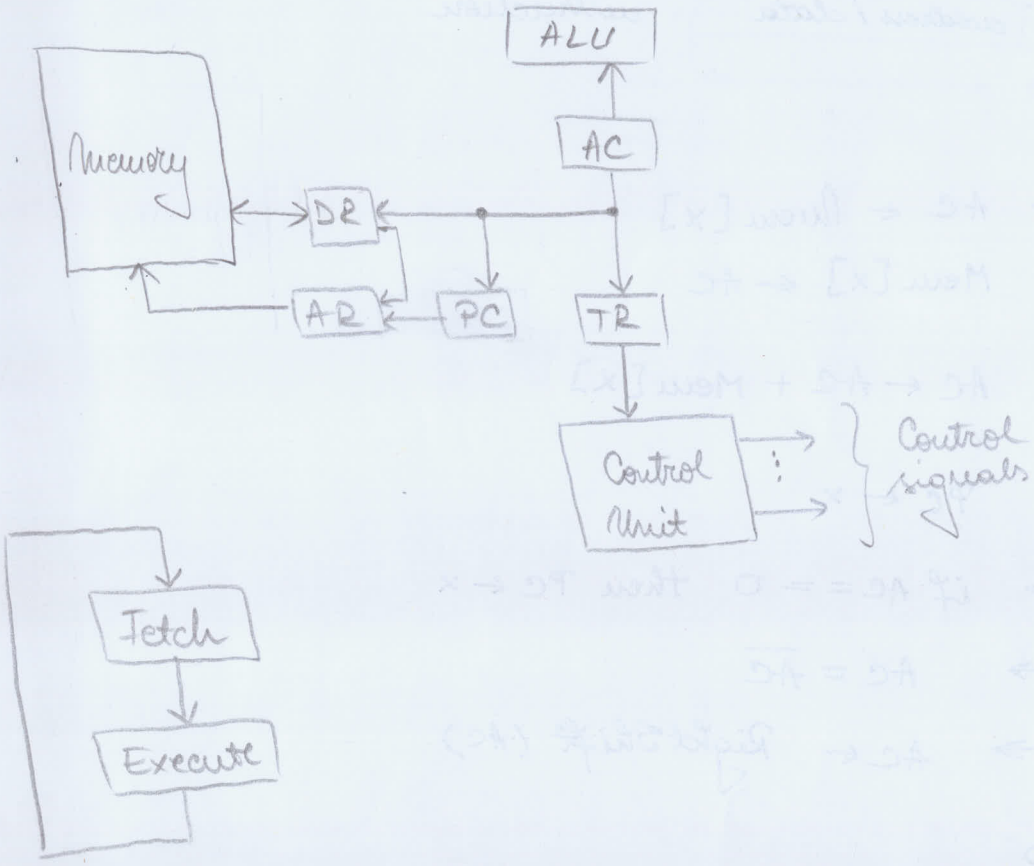


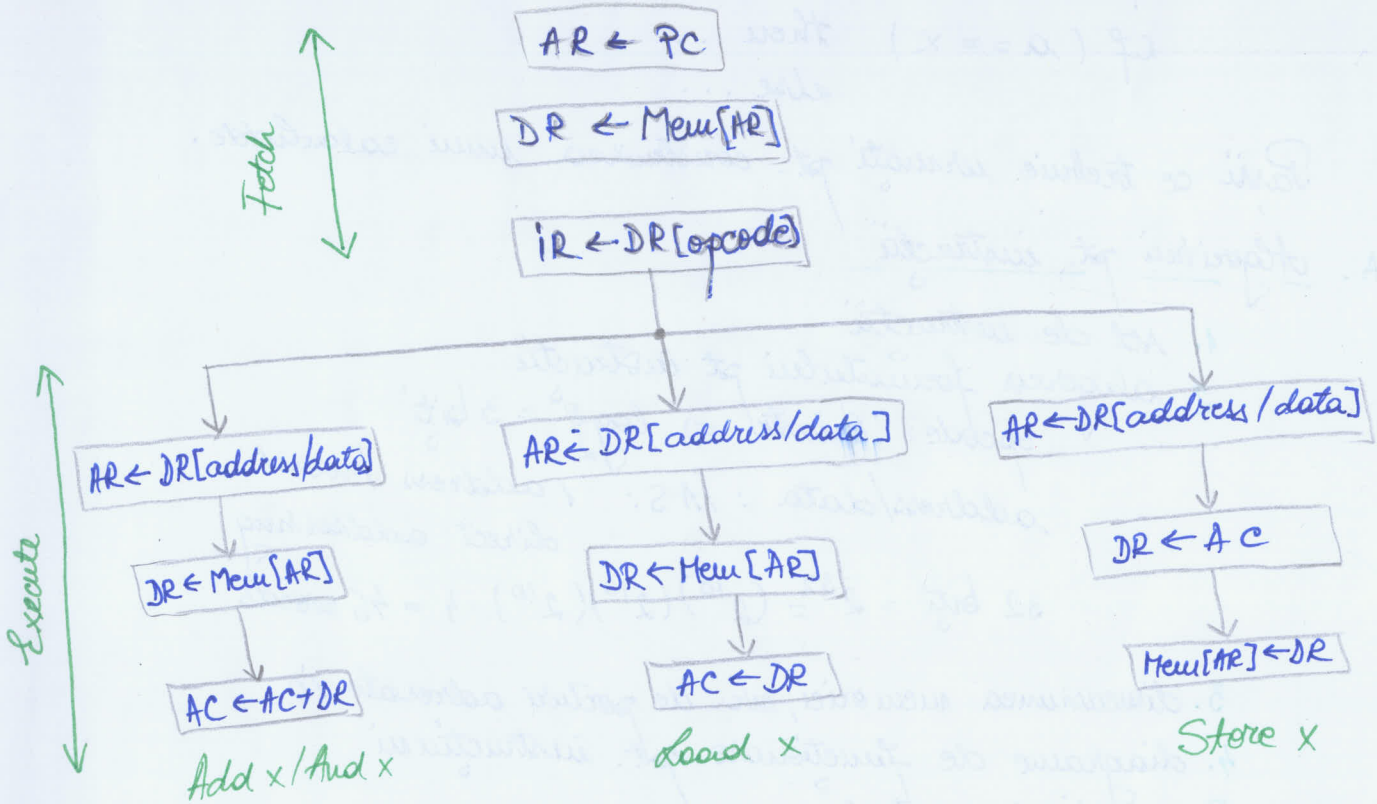
load next instruction in IR.

Fetch instruction



CURS 2 - 7.10.2016.





word | value | data

word | opcode | address / data | instruction

? ?

Load x ⟶ AC ← Mem[x]

Store x ⟶ Mem[x] ← AC

Add x, And x ⟶ AC ← AC + Mem[x]

Jmp x ⟶ PC ← x

Jmpz x ⟶ if AC == 0 then PC ← x

comp ⟶ AC = $\overline{AC}$

Rshift ⟶ AC ← RightShift (AC)

a - x == 0

⋮

if (a == x) then ...
else ...

Pașii ce trebuie urmați pt. construirea unui calculator.

A. Algoritmi pt. instrucțiuni

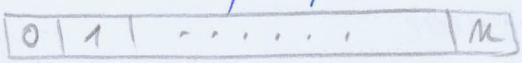
1. set de instrucțiuni
2. alegerea formatului pt instrucțiuni
   opcode: 8 instr ⇒ $\log_2 8 = 3$ biți
   address/data: IAS: 1 address / word
   direct addressing
   32 biți = $2^{32} = (2^{10})(2^{10})(2^{10}) \cdot 4 = 4G$ words
3. dimensiunea memoriei, nr. de porturi adresate I/O.
4. diagrame de funcționare pt. instrucțiuni
5. unități de control.

B. Alegerea tehnologiei în vederea implementării

C. Implementare

1.2. Setul de instrucțiuni

Orice instrucțiune conține 2 sau mai multe câmpuri, din care unul e dedicat pt opcode, iar de la 1 → n / operanzilor.



Operanzii pot conține valori sau adrese. Adresele pot fi de memorie sau de port. Fiecare instr. efectuează o operație ce este o funcție de cei n operanzi. Specif. celor n op aduce la o lungire inacceptabilă a cuvântului de instrucție, inf. asupra spațiului de memorie ce a dus la un nr minim de operanzi.

$f(x_1, x_2, \ldots x_n)$

$m < n$

↓

m address machine

m – nr max. adrese, restul fiind implicit considerate adresă de registrii.

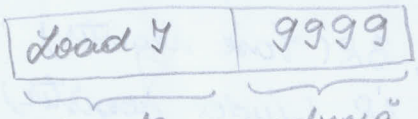
1.3. Moduri de adresare

Unui câmp de adrese / operand. i se asociază într-un anumit fel o valoare, iar modul în care poate fi folosită sau specificată această adresă se numește mod de adresare.

a) adresare imediată:

- adresa este dată în mod explicit ca și o val const.

ex: Load I 999 (load imediat)



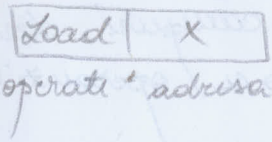
constante → nu pot fi modificate.

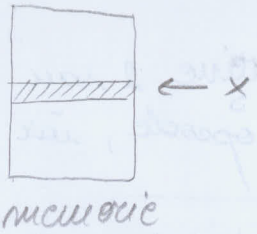
- un acces la memorie; nu există flexibilitate

b) adresare directă

- similară cu a), dar adresa este variabilă și poate fi specif. de utilizator; 2 acces la memorie

ex: Load x

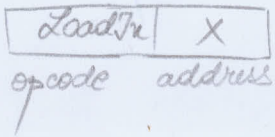


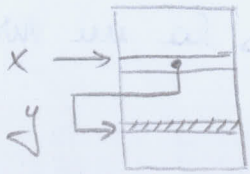


c) adresare indirectă

- x este un pointer la o altă adresă, care conține datele ce trebuie transferate ; 3 accese la memorie
- flexibilitate maximă.

ex: LoadIn x





Ortogonalitate = abilitatea unui procesor de a realiza toate modurile de adresare.

1.4. Tipuri de adresare.

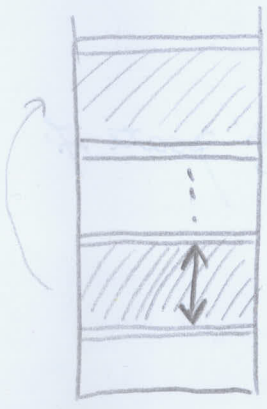
a) adresare absolută

- adresa este pe toată lungimea ei (toată lung. câmpului rep. o adresă)

b) adresare relativă

- pt lucrul la nivel mai înalt
- adresa = val. specificată la un deplasament
- există 2 modalități
  - cu registru de bază, BR (base register)
  - cu registru index, IR (index register)

adresarea cu BR - apare când se dorește relocarea unei zone de memorie în alt loc / altă zonă.



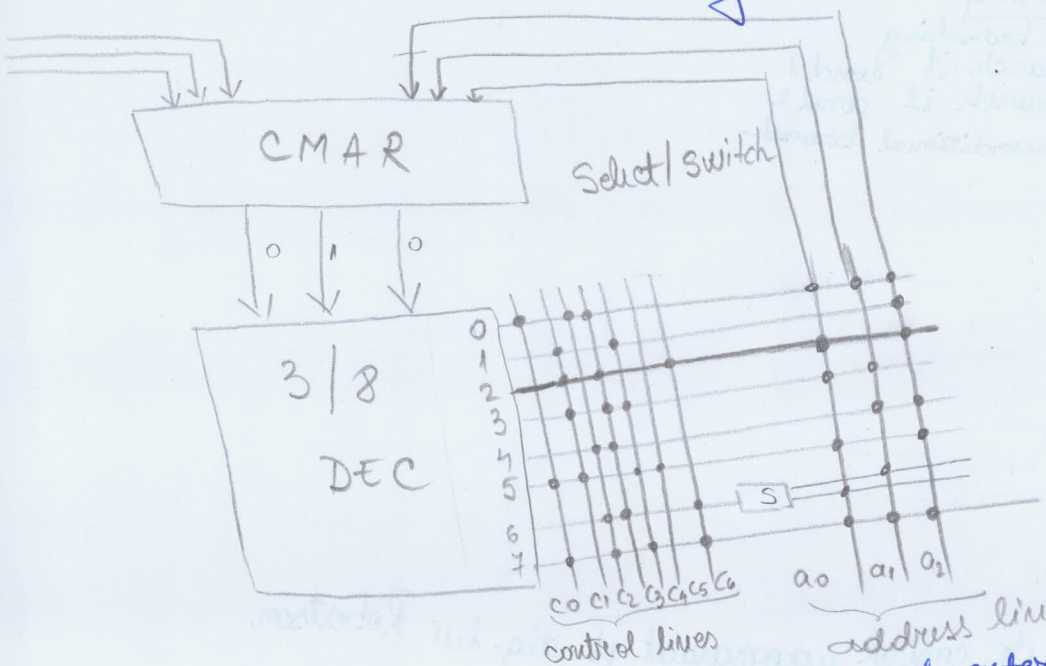
address = valoare + BR,
valoare ∈ [0,99]

$BR_{init} = 100$

$BR_{final} = 1000$.

2.3. Proiectul lui Wilkes.

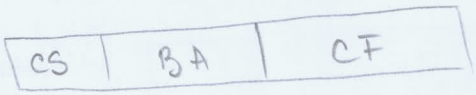
Sir Maurice Wilkes, 1951 Cambridge.



S → permite comanda adresei în fct. de o cond. externă.

CMAR → poate fi încărcat de la o sursă ext, dar și din surse gen. interne, fiecare linie act. de DECODIFICATOR având drept ef. activarea unor linii de adresă și a liniilor de control.

2.4. Organizarea unei unități de control programabile.

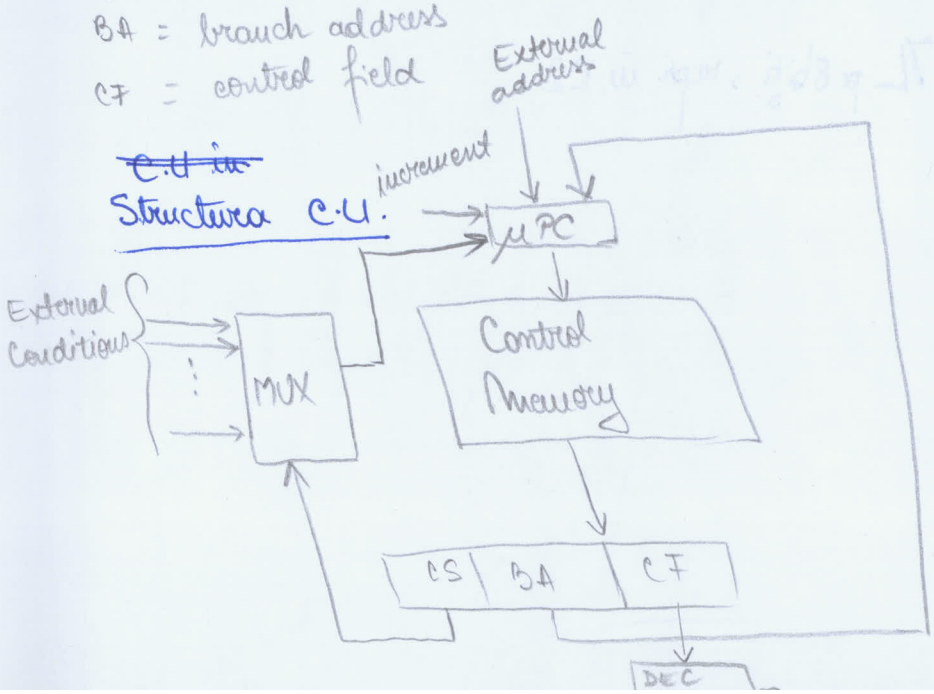


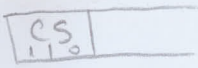
CS = condition select
BA = branch address
CF = control field

(*) dec. generează semnale de control

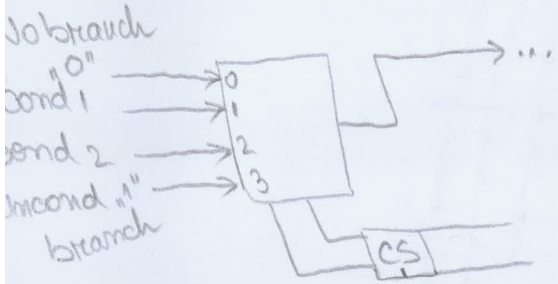
~~C.U în~~

Structura C.U.

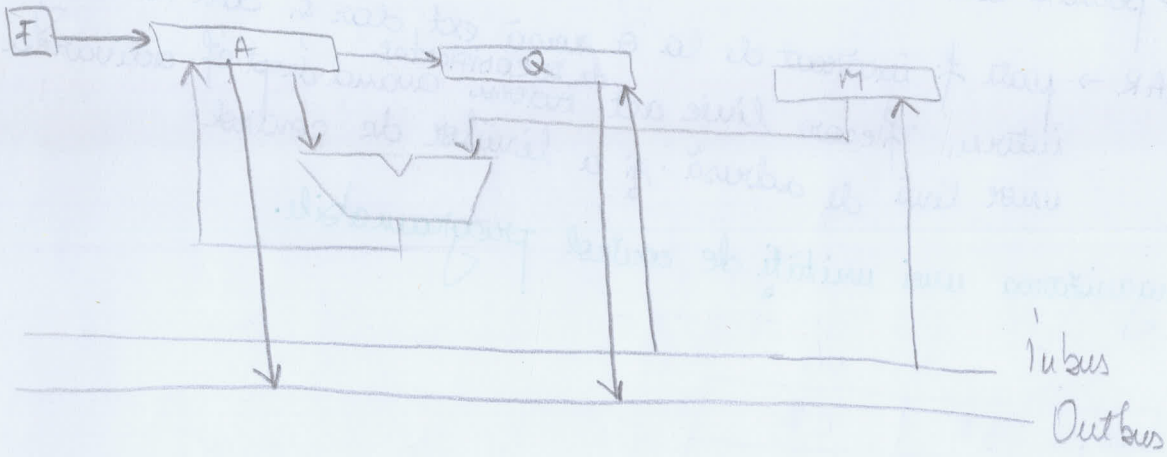




CS	Meaning
0 0	No branching
0 1	Branch if cond 1
1 0	Branch if cond 2
1 1	Unconditional branch



2.5. Unitate de control μprogramat pt. alg. lui Robertson



Considerăm operanzi în C2 pe 8 biți, repr. în C2.

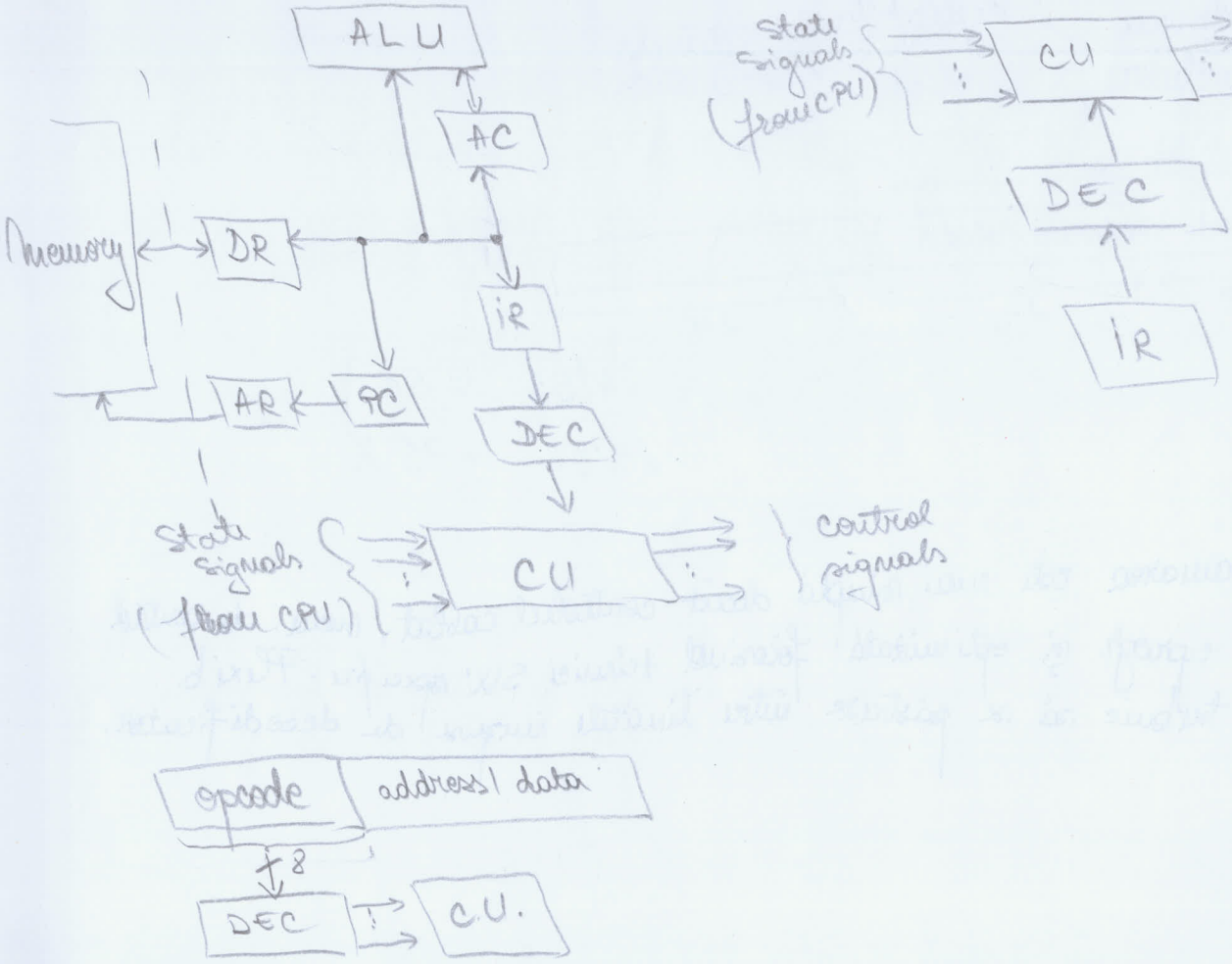
CURS 3 – 14.10.2016.

II. Designul și implementarea unităților de control.

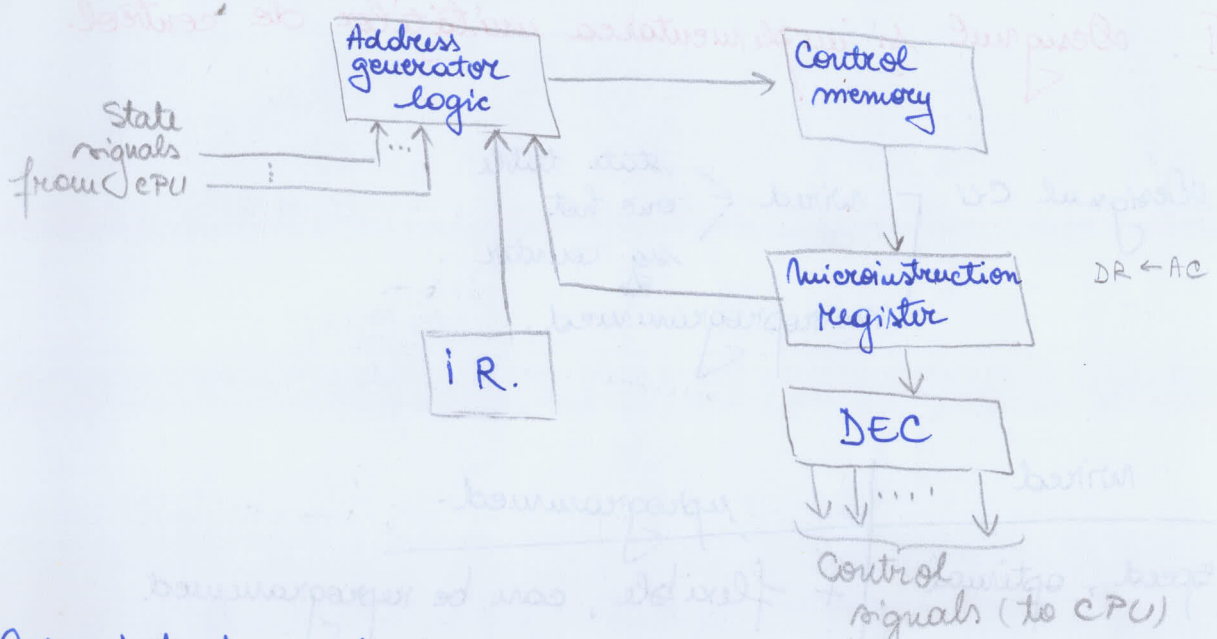
Designul C.U
- wired (cablat)
  - state table
  - one hot
  - seq. counter
  - ≅ HW
- microprogrammed. ≅ SW

wired	microprogrammed
+ speed, optimal	+ flexible, can be reprogrammed
– not flexible	– not so fast

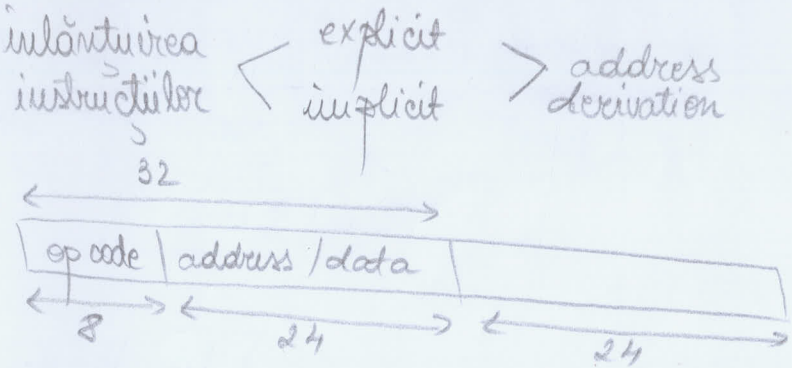
2.1. Hardwired control units



2.2. Microprogrammed control units.



Orice instruct. din IR det. un fetch în mem. de control la adresa care marchează începutul secv. de microinstr. care implementează acea instr. Această secv. va det. decodif. repetate și deci generarea de semn. de control potrivit.



24/56 ≅ 50%

24/32 = 75%

- microprogramarea este mai simplă decât controlul cablat, mem. de control poate fi reprogr și optimizată folosind tehnici SW specifice. Flexib. soluția trebuie să se păstreze între limitele impuse de decodificator.

ex: read (x1);
read (x2);
PC → print (suma (x_1, x_2));
:
PC → suma (a, b)
{
a+b;
return; → refac stiva (a lui PC și a alter reg.) cu valorile salvate în stivă înainte de print.
}

P1

P2

FIC - curs 4
21. 10. 2016.

Begin: A ← 0, F ← 0, count ← 0, M ← Intbus; {c1}
Q ← Intbus {c2}

Test 1: if Q[0] = 0 then go to RightShift {c3}
A ← A + M, F ← F + M[7] {c4}

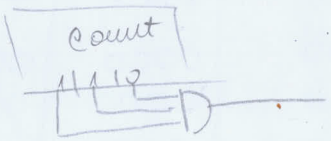
RShift: A.Q ← FA.Q[7:1]; {c5}
if count 6 = 1 then go to Test 2;
count ← count + 1, go to Test 1

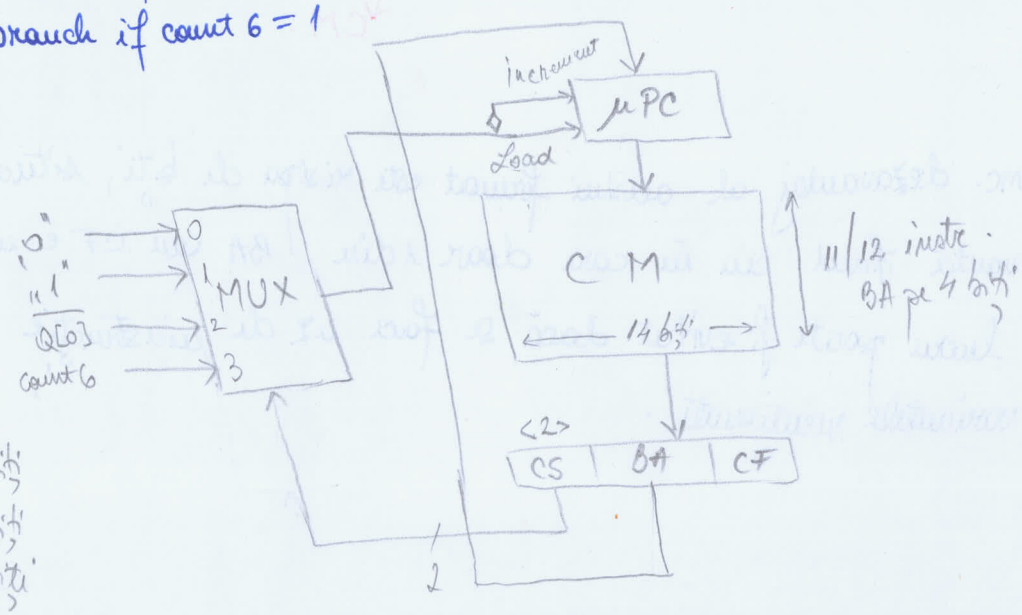
Test 2: if Q[0] = 0 then go to Output {c3, c6}
A ← A - M

Output: Outbus ← A; {c7}
Outbus ← Q; {c8}

End: go to End.

CS:	
0 0	No branching
0 1	Unconditional branch.
1 0	Branch if Q[0] = 0
1 1	Branch if count 6 = 1





CS: 2 biți
BA: 4 biți
CF: 8 biți

Câmpul de control poate recurge la diverse codificări dintre care cele mai cunoscute sunt:

encoding by function

BA or CF

sau poate să nu utilizeze nici un câmp de codificare, caz în care formatul instrucțiunii se numește microprogramare pur orizontală.

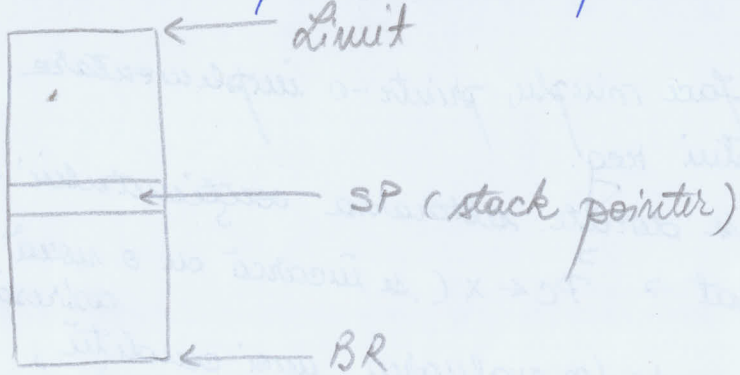
Fm cont. vom ex alg lui Robertson în format pur orizontal.

Address în CM	CS	BA	CF
0000	00	0000	10000000
0001	00	0000	01000000
0010	10	0100	00000000
0011	00	0000	00100000
0100	00	0000	00010000
0101	11	0111	00000000
0110	01	0010	00001000
0111	10	1001	00000000
1000	00	0000	00100100
1001	00	0000	00000010
1010	00	0000	00000001
1011	01	1011	00000000

→ CM.

Princ. dezavantaj al acestui format este risipa de biți, situația dominantă fiind cea în care doar 1 din BA sau CF e utilizat. Acest lucru poate fi evitat dacă se face uz de minstrucție în una din variantele menționate.

BR se mai foloseste si la op. cu stiva



$BR \leq SP \leq Limit$

operatiile cu stiva < push
pop.

adr. cu Ri - se foloseste la accesul unei secv. de cuvinte

x[0], x[1], x[2], ...

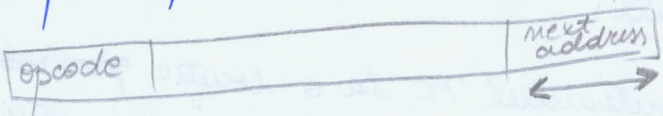
value ↑ index

Este suf. să cunoaștem adresa primului elem. și distanța între x[0], x[1], numită index.

Concluzie: apare posibilitatea autoindexării

1.5. Înlănțuirea instrucțiilor (instruction chain)

- cum det. care e urm. instr. pe care o voi executa, știind-o pe cea curentă
- folosim registrul special: program counter (PC)
- metoda simplă: specific în cuvântul de instr adresa urm. instr.



* dezavantaj: dimensiune prea mare a cuv. de instr.

- altă metodă: utilizăm Program Counter (PC)
- rostul reg. PC constă în faptul că de cele mai multe ori exec. unei instr. i din- un prog. este urmată de exec. unei instr $i+1$.

- adresa lui i+1 se det. din adr. lui i + dim cuv de memorie
- adresarea prin PC se face simplu, printr-o implementare cu incrementare a acestui reg.
- există 2 s/t. în care se dorește alterarea conținutului lui PC
  - salt necondiționat → PC ← X (se încarcă cu o nouă adresă)
  - salt condiționat → are loc evaluarea unei condiții, indicată prin fanioane dintr-un reg. special de stare, Status Register (SR); fanioane: zero, carry, sign, ...
- apare de asemenea s/t. când controlul se dorește transf. de la prog. P1 aflat în exec., către o fct. (progr.) P2
- transf. controlului de la P1 la P2 apare în 2 situații:
  - apelul de funcție / procedură
    call procedure
    call routine
  - când cedarea controlului e inițiată de P2 sau a dispozitivului asociat lui P2, prin trimiterea unui semnal către procesor, numit întrerupere

→ de regulă, prezența lor e verificată la sfârșitul fiecărei instr.

O instr. de tip call:

1. se salvează conținutul PC la o locație ~~predefinită~~ predestinată (reg. special / zonă de memorie utilizată de regulă ca o stivă)
2. PC se încarcă cu adresa pr. instr. a rutinei, începând execuția acesteia

- Encoding by function

O codificare maximală a câmpului de control ar duce la situația în care semnale care nu au o corelare funcțională, participă în același câmp. Această situație rep. un dezavantaj în cea ce privește depanarea microcodului

Se poate constata că struct. microcodului se bazează pe funct. unor componente majore:

- sumator
- magistrală de in
- reg. de out.
- count
- A.Q

O asemenea perspectivă duce la discuții cu privire la struct. pe componente a sistemului.

Exemplu:

Addr	00	No operation
	01	A ← A+M , F ← A+M[7]
	11	A ← A-M , F ← A+M[7]
	10	Unused
Inbus	00	No op.
	01	M ← Inbus
	10	Q ← Inbus
	11	Unused
Outbus	00	No op
	01	Outbus ← A
	10	Outbus ← Q
	11	unused
shift	0	No op
	1	A.Q ← F.A.Q[7:1]
count	00	No op
	01	count ← 0 , A ← 0 , F ← 0
	10	count ← count + 1
	11	unused.

	Address in CM	CS	BA	
Begin:	0000	00	0000	0001 00 001
	0001	00	0000	00 10 00 0 0 0
Test 1	0010	10	0100	0000 0 0 0 0 0
	0011	00	0000	0 1000000 0
RShift	0100	00	0000	0000000 1 0
	0101	11	0111	00 0000 1 00
	0110	01	0010	0000000 1 0
Test 2:	0111	10	1001	0000000 0 0
	1000	00	0000	11 0000000
Output:	1001	00	0000	00 0001000
	1010	00	0000	00 00 10 000
	1011	01	1011	0000000000

Deseu tabă. Ca al de pe pr. pag, ar cere în plus la CS BA CF

• BA or CF.

– presup. rescrierea alg. a.i. fiecare instr. să exec. fie un salt fie să activeze unul semnal de control, dar nu ambele!

Count ← count+1, go to Test1; → { count ← count+1; go to Test1;

Tehnica nu presup. modificări ult. ale CF sau

Ex. pt. rescrierea alg. ; a μprog, fără a utiliza vreo modif. asupra CF.

	Address în CM	CS	BA or CF c_1 c_2 c_3 c_4 c_5 c_6 c_7 c_8 or a
Begin	0000	00	10000000
	0001	00	01 00 00 00
T1	0010	10	0000 0100
	0011	00	0010 0000
Rshift	0100	00	0001 0000
	0101	11	0000 1000
	0110	00	0000 1000
	0111	01	0000 0010
T2:	1000	10	0000 1010
	1001	00	0010 0100
Output	1010	00	0000 0010
	1011	00	0000 0001
End:	1100	01	0000 1100.

1: 12 x 14 = 168 biți

2: 12 x 15 = 180 biți

3: 13 x 10 = 130 biți

FIC - curs 5

28.10.2016.

Begin : A:=0, count ← 0, F ← 0, M ← Inbus {c1}
Q ← Inbus {c2}

Test 1 : if Q[0]=1 then go to RShift {c3} → incarcă rezultatul
A ← A+M, F ← F+M[7];

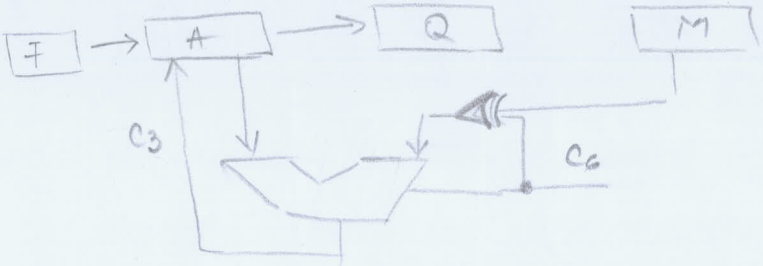
RShift: A.Q ← F.A.Q[7:1] {c4}

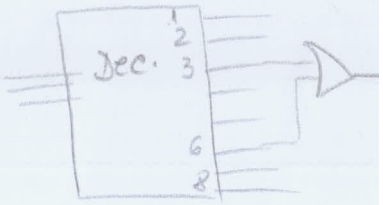
Test 2 : if count6=1 then go to Test 3; {c5}
count ← count+1, go to Test 1;

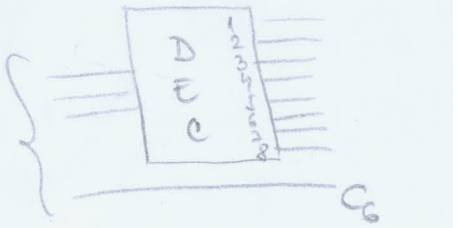
Test 3 : if Q[0]=0 then go to Output; {c3, c6}
A ← A+M;

Output : Outbus ← A ; {c7}
Outbus ← Q ; {c8}

End : go to End







{c6}, {c1, c2, c3, c4, c5, c7, c8}
<1> <3>

	CS	BA & CF
Begin 0000	00	0 001
0001	00	0 000
	01	0 100
	00	0 011
	00	0 100
	10	1 000
	00	0 101
	11	0 010
	01	1 010
	00	1 011
	00	0 111
	00	0 110
	11	1 100

BA2:4	
000	
001	C1
010	C2
011	C3
100	C4
101	C5
110	C7
111	C8

BA1	
0	C6 NOT ACT
1	C6

Codif. maxim. a s. de control poate fi utilizată cât vreme nu ∃ . În caz contrar se impune fie prefixarea logică suplimentară, fie extinderea lg de biți pt. codificare

Abordarea noastră → câmp suplimentar pt CG

Metoda BA or CF presupune rescrierea codului pt a elim. s/t. pt care se exec. un salt simultan cu activ. unor s. de control.

CURS 6 - ACC
04. 11. 2016.

III Pipelining

Pipelining = suprapunerea în timp a execuției instrucțiunilor.

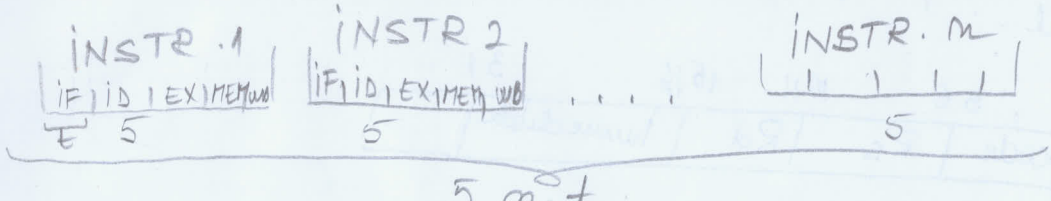
ADD R_5, R_6, R_7

MUL R_1 R2 R3 <=> $R_1 \leftarrow R_2 * R_3$

Exec. unui prog = exec. secv. a instr. în cinda exist. paralelismului care există la nivel de operații. Ideea fiecare instr rep. o secv. de operații apare ca fiind posibilă suprapunerea exec. instr. cu cond. ca fiecare să se afle într-o fază dif. a exec.

Ex. vom consid. setul de instr și arh. lui DLX.

DLX → pipeline pe 5 stagii
- IF - instr. fetch
- ID - instr. decode
- EX - execute
- MEM -
- WB -



INSTR 1 : IF ID EX MEM WB 5t.

INSTR 2 : IF ID EX MEM WB 6t

⋮

INSTR n : . . . . . (n+4) t

$$\text{Speed up} = \frac{\text{Time - no - pipeline}}{\text{Time - pipeline}} = \frac{5nt}{(n+4)t} = \frac{5n}{n+4} \to 5.$$

INTEL 80486. ⟶ 5 stagii

ARM → smartphone ⟶ 3 stagii

Dacă nu este suficient de mare ca să obț. accelerări teoretice de val. nr de stagii. Pt a putea înțelege mec. care intervin vom aj. versiunii non-pipeline a procesorului DLX.

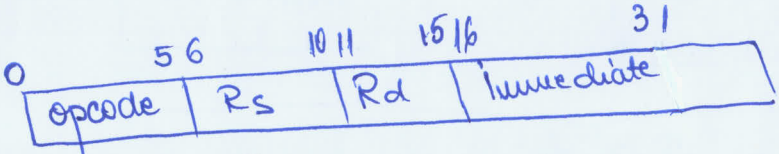
~~3.1. Non pipeline DLX.~~

3.1. Non pipeline DLX (~~Patterson~~ & Hennessy)

DLX e un proc. spl. care impl. o mașină de tip load-store cu 32 registrii și tot 32 de reg pt ver în virg. flotantă.
(F0 ÷ 31 : $F_{0,1}$, $F_{2,3}$ ... $F_{30,31}$).
Există 4 tipuri de instr.: load-store
Jump / Branch.
ALU
Floating Point.

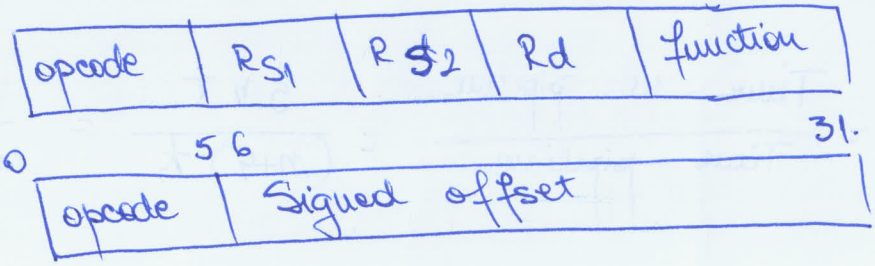
SAU INSTR { registru / jump / immediate.

Instr. sunt repr. pe 32 biți, iar unit. de mem. adresabilă este octetul.



$2^6 = 64$ posibilități; $2^5 = 32$

Add R_3 R_2 #5 (=) $R_3 \leftarrow R_2 + 5$.



1. În stagiul de IF:

- aduc din mem. instr. Prog Counter (PC) care îmi dă adresa următ. instr

IR ← Mem[PC]

NPC ← PC + 4 → mărimea unei instrucțiuni în bytes (unit. adresabilă e octetul)

new PC ←

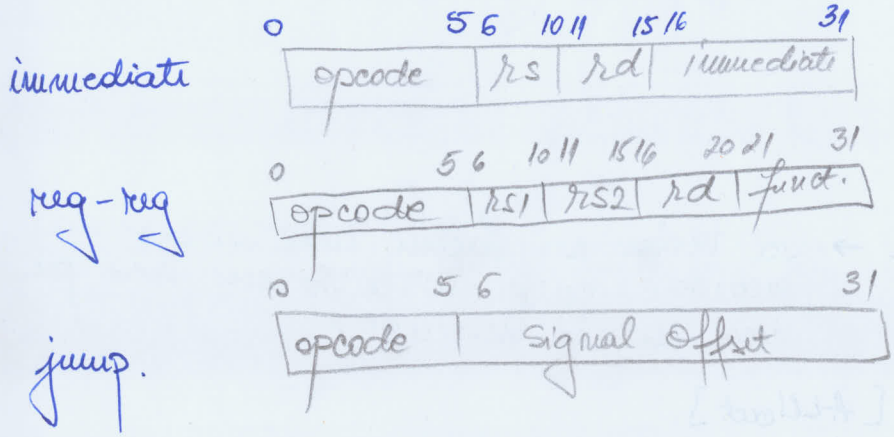
2. ID

FIC - CURS 7.

11. 11. 2016.

3. Pipelining

3.1. DLX non-pipeline



IF, ID, EX, MEM, WB.

- **IF**: IR ← Mem[PC]
  NPC ← PC + 4
- **ID**: A ← Regs[IR6:10]
  B ← Regs[IR11-15]
  Imm ← IR16^16 ## IR16-31

  rd va fi updatat în faza de WB.
- **EX**:
  - mem-reference
    ALUout ← A + Imm
    ex: LW R1, 40(R3) (=) R1 ← Mem[40+R3]
    SW 40(R3), R1 (=) Mem[40+R3] ← R1
  - register-register:
    ALUout ← A function B.
    ADD R1, R2, R3 (=) R1 ← R2 + R3
  - ALU immediate
    ALUout ← A opcode Imm.
    ADD R1, R3 ## 40 (=) R1 ← R3 + 40.
    MUL R1, R3, #40 (=) R1 ← R3 * 40.

- branch:

  $ALU_{out} \leftarrow NPC + Imm$

  condition $\leftarrow$ A opcode '0'

if $(x < y)$ then secv 1
else secv 2. $\Leftrightarrow$ $x - y < 0$.

- MEM:
  - memory reference → va trebui să facem acel acces la memorie care să aibă ca rezultat pe care să îl memorez.

    $LMD \leftarrow Mem[ALU_{out}]$.

    $Mem[ALU_{out}] \leftarrow B$.
  - branch

    if condition then $PC \leftarrow ALU_{out}$
    else $PC \leftarrow NPC$.

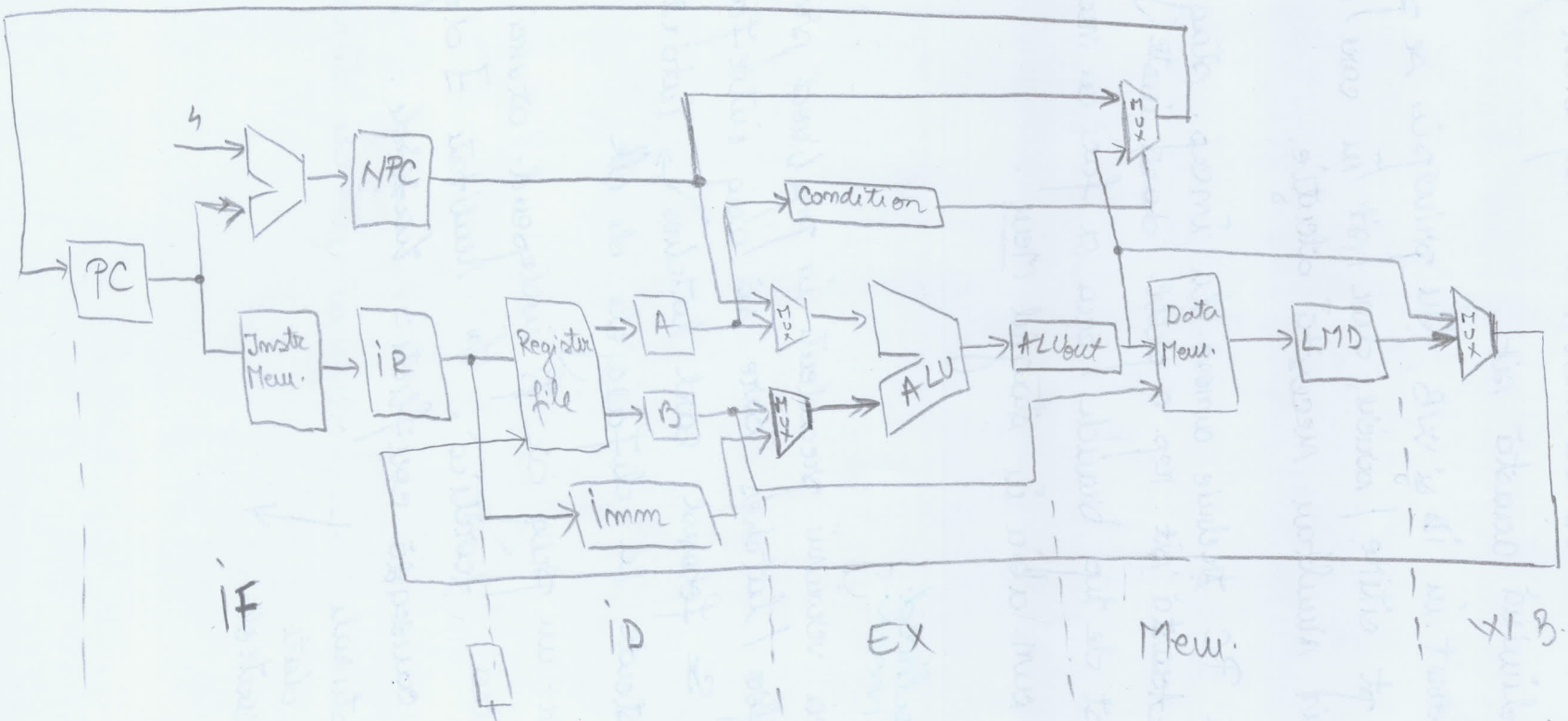
- W/B
  - register - register

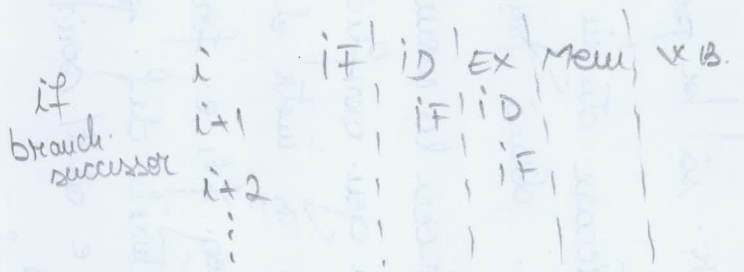
    $Regs[IR_{16 \div 20}] \leftarrow ALU_{out}$
  - ALU - immediate

    $Regs[IR_{11 \div 15}] \leftarrow ALU_{out}$
  - Load

    $Regs[IR_{11 \div 15}] \leftarrow LMD$ (load memory data)

RISC vs CISC





toate mă trebuie memorate, de asta au nevoie de buffere

Reguli:

(1.) Pt ca pipeline-ul să funct. e necesar ca (∀) comb. de instr. să se poată exec. fără apariția unor conflicte

(2) Fiecare stagiu trebuie să fie activ în orice comb. de instr. Dacă nu, secv. de instr va fi întârziată.

(3.) Accesa la memorie au loc simultan în IF și Mem. și poate gen. conflicte. Folosirea unor mem. separate pt. date și instr elimină această sit

(4) Reg. file e folosit în ID și WB. În principiu se pot folosi fronturile dif. pt citire / scriere, dar sit în care același reg e citit/ scris simultan necesită atenție.

(5.) Noua val pt PC trebuie anunțe. la începe. stagiului de fetch. Această sit. rep. o probl. dacă instr anterioară a fost de tip branch / ccea a face ca noua val pt PC să fie cun. abia în stagiul Mem.

3.2. DLX - pipelined

Transformarea versiunii precedente în pipelined presup exist. unor buffere / latches care să asig izolarea dintre stagii. Se folosesc earl latches → întârziere const. și rezistente la defazajele de clk.

Dacă instr dintr-un prog ar fi independ. atunci perf. pipeline. ar fi cea teoretică. În realitate ∃ depend. între instr. care cauzează conflicte = hazarde.

Hazarde ↑ structurale
de date
de control. ↓ → cele mai usor de stăpânit

5. Hazarduri:

Hazard structural → conflict între resurse.

- apare când d.p.d.v. HW nu pot fi suportate toate combinațiile.

ex. mem la i și IF la i+3.

Hazard de date → când ex. unei instr. dep. de exec. instr. anterioare. astfel că exec. acestia nu poate continua.

FIC – curs 8 – 18.11.2016.

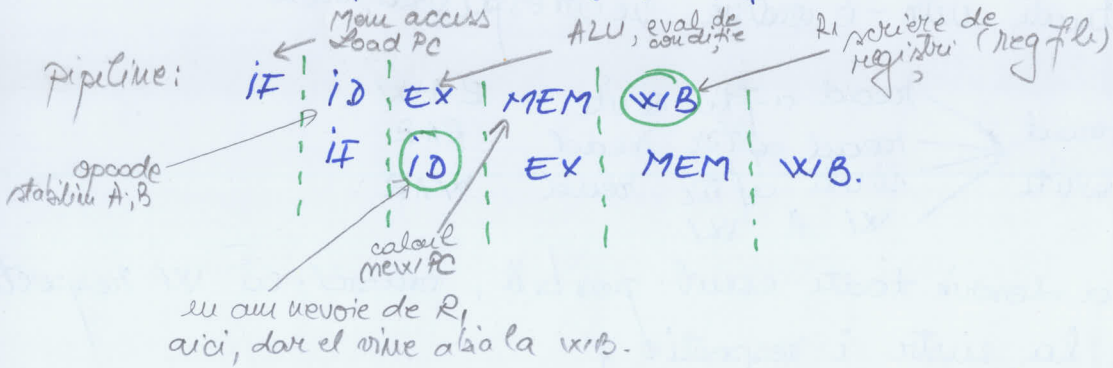
S12. – Miercuri curs FIC ora 12:00 (14 dec.)
S9 – Joi curs FIC ora 12:00 (24 noiembrie)

Hazard ← structural
data
control.

Hazard de date – apare în momentul în care există comb. de instr. în care operanzii unei instr. depind de rez. altei instr. (când o instr. are nevoie de rezultatul produs de una din instrucțiunile precedente)

exemplu:

ADD R_1, R_2, R_3 (=) $R_1 \leftarrow R_2 + R_3$
SUB R_4, R_1, R_5 (=) $R_4 \leftarrow R_1 - R_5$



Instrucția ADD actualizează val lui R_1 în stagiul WB, dar această val e necesară pt instrucția SUB în stagiul ID. Dacă nu se iau măsuri, instr. SUB va citi o val corectă (val nu e nici măcar deterministă).

Soluția poate fi mai simplă sau mai elaborată, implicând penalizării dif. de perf.

Sol. A: e necesar un mix de HW pt a detecta dependențele.

a) Dacă există, pipeline-ul se oprește. (pipeline freezing / stop)

ADD R_1, R_2, R_3 IF ID EX MEM WB.
SUB R_4, R_1, R_5 IF ID stall stall EX MEM WB

* stall / bubble → se propagă în timp

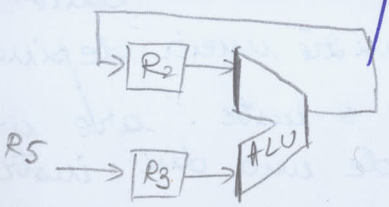
b) forwarding / bypassing.

Rez. de la ieșirea ALU poate fi întotdeauna dus în buff. sau latch-urile de la intr. ALU. Dacă se detectează că instr. următ. va scrie într-un reg. din instr. curentă, atunci logica de control va sel. rez. ALU și nu val. citită din registru.

ADD R_1, R_2, R_3 — IF ID EX MEM WB (R_2, R_3 la ID; R_1 la WB)

SUB R_4, R_1, R_5 — IF ID EX MEM WB.

– nu vom avea nici o penalizare.



Hazardul de date poate fi clasificat după comb care îl generează, respectiv comb. de scrieri, citiri, care sunt efectuate într-o ordine necorespunzătoare.

- read
- write

← read after write — RAW
← read after read — RAR
← write after read — WAR
WAW

Pt a descrie toate comb. posibile, cauzid. că WI respectiv R apar la instr. i respectiv j

RAW → apare când instr j încearcă să citească o sursă înainte ca i să o scrie → j va citi val. veche, eronată
→ cel mai comun tip de hazard și e principalul beneficiar al sol. forward and bypassing.

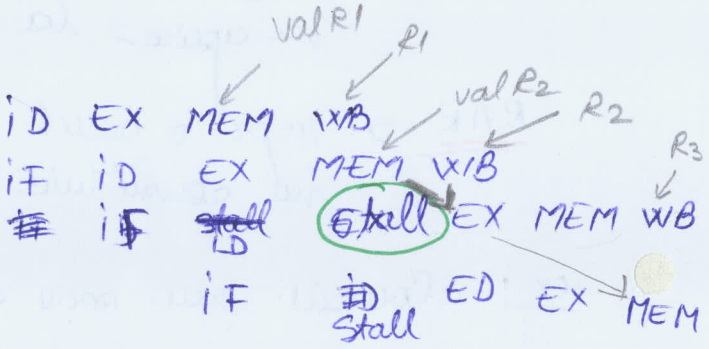
~~RAR~~.

WAW → instr j încearcă să scrie un operand înainte ca scrierea să fie ef. de instr. i => vom avea o succesiune de 2 scrieri cu ordinea incorectă, val finală nefiind cea corectă.
→ procesorul DLX nu are acest hazard, d-ce sing. stadiu în care sunt scrieți regiștrii e WB.

În această sit. oprirea pipeline e inevitabilă. Mec. de FW/ bypass neputând rezolva. faptul că val necesară nu e propriu de la ALU ci trebuie cit. din mem. Această sit. se num. pipeline interlock. și introd. o penalizare de 1 ciclu de clock. Din fericire aceste sit pot fi evitate cu ajutorul compilatorului

Considerăm urm. ex:

```
LW   R1, B
LW   R2, C
ADD  R3, R2, R1
SW   A, R3.
```



A ← B + C

- au pierdut un c.c.

În această sit. a calc. tipic al unei sume, sit. de pipeline interlock nu poate fi evitată, de multe ori însă se dorește ef. a 2 calc. de genul A ← B+C, urmată de D ← E−F.

```
LW   Rb, B
LW   Rc, C
LW   Re, E
ADD  Ra, Rb, Rc
LW   Rf, F
SW   A, Ra
SUB  Rd, Re, Rf
SW   D, Rd.
```

→ în sist. moderne instrucț. sunt pipelined. Este posibil ca pentru anumite instrucț. finalizării ~~ale~~ instr. să apară acest tip de hazard.

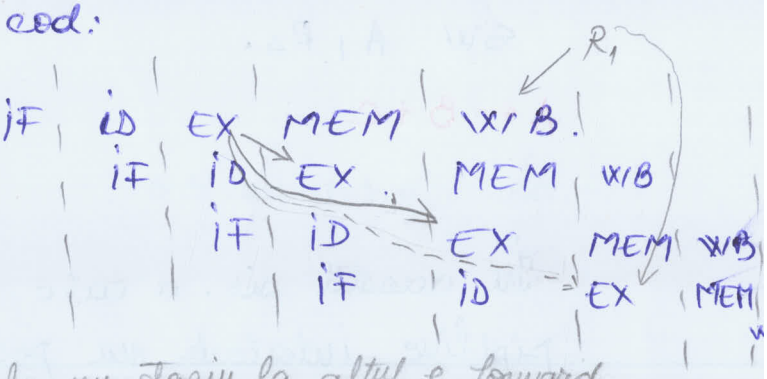
W/AR → când i₂ scrie un reg. destinație înainte ca acesta să fie citit de i₁ (i₁ < i₂) instr. i₁ va citi o val. eronată.

→ d-ce toate citirile se fac devreme în stagiul ID și toate scrierile târziu în W/B acest hazard nu apare la DLX.

RAR → repr. o comb. care nu alterează date și deci nu constituie hazard.

ex: Consid. urm. secv. de cod:

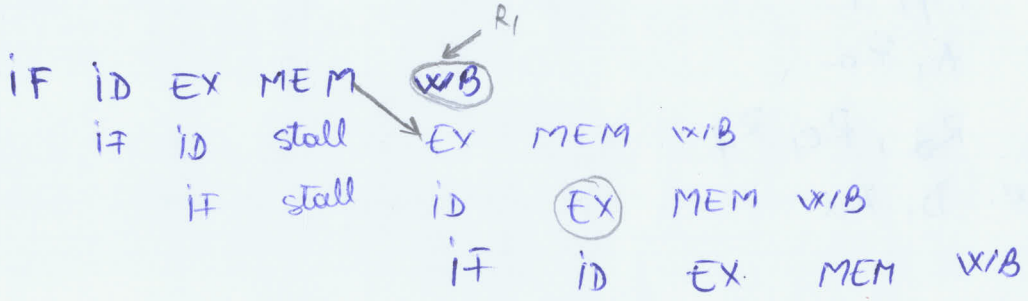
```
LW   R1,30(R2)
SUB  R4, R1, R5
AND  R6, R1, R7
OR   R8, R1, R9
```



- de la un stagiu la altul e forward
- dacă e nevoie de mai multe stagii, e bypassing

Situația ilustrată ar presupune o spl. iterație prin care rez. ALU este forwardat la intrările ALU pt a preîntâmpina.

ALU. Totuși această sit. doar prin mec de F W/AR și incorect gestionată de-ce în EX al instr. LW se calc. adresa



IF ID EX MEM WB

IF ID EX MEM WB

IF ID EX MEM WB.

IF ID EX MEM WB val Rt ↙ Rf.

IF ID EX MEM WB.

IF ID EX MEM WB.

IF ID EX MEM

IF ID EX

FIC – curs 9.
24.11.2016.

Delayed load

O instr. de tip load care cere ca rez. sau să fie utilizat de instr. urm. se numește delay load. Poz. din pipeline de după delay load se num. delay slot.

Compilat poate elim. maj. dependențelor delay load, fiind posibile 4 situații:

- fără dependență → inst care urmează după load nu folosește rez. acesteia.

```
LW  R1, 30(R2)      R1 ← Mem[30+R2]
ADD R3, R4, R5      R3 ← R4 + R5
SUB R6, R7, R8
AND R9, R10, R11
```

- dependență cu oprire →

```
LW  R1, 30(R2)      IF ID EX MEM WB
ADD R5, R1, R7         IF ID stall EX MEM WB
SUB R8, R7, R6            IF stall ID EX MEM WB
AND R9, R6, R7                   stall IF ID EX MEM WB
```

(30+R2, Mem[MW], R1)

* MUX în care o instr. părăsește stagiul ID și intră în EX. Repr. un MUX care afectează starea internă a procesorului ID/EX → instruction issue.
- pt detecția acestor dependențe se folosesc niște comparatoare care vor compara cei 2 surse de la instr. curentă cu cei 2 dest. date de cei 2 instr. anterioare. În acest caz va fi necesar un ciclu de stall (de oprire).

- dependențe depășite prin forwarding

```
LW  R1, 30(R2)      IF ID EX MEM WB
ADD R5, R6, R7         IF ID EX MEM WB
SUB R8, R7, R1            IF ID EX MEM WB
AND R9, R6, R4               IF ID EX MEM WB
```

(Mem[MW], R1)

• dependente cu acces ordonat.

```
LW  R1, 30(R2)     IF  ID  EX  MEM  W/B.
ADD R5, R6, R4         IF  ID  EX   MEM  W/B
SUB R8, R7, R6             IF  ID   EX   MEM  W/B
AND R9, R6, R1                 IF  ID   EX   MEM  W/B.
```

(R1 — săgeată spre W/B)

- în decode se decodif. reg sursă.

→ în această sit. nu e necesară nici o acț. specială d-ce cît. lui R1 în AND se întâmplă în cea de-a 2 parte a lui ID, pe când scrierea lui R1 în load are loc devreme în W/B.

Hazardul de control.

→ Hazardul de control poate cauza pierderi mai mari de perf. decât haz. de date.

→ apare dat. faptului că o instr. de salt condiționat poate modifica adresa în prog. counter (PC), cu altă val. decât PC + 4.

→ comportamentul unui branch este: ↳ PC ← PC+4 (branch not taken)
↳ PC ← new value (target branch, branch taken)

→ calculul valorii noului PC este cunoscut în stagiul MEM.
Spunem că în acest stadiu are loc rezolvarea branch-ului (branch resolve), ceea ce înș. că până în acel mom. pipeline-ul ar trebui oprit. Această tehnică este cea mai simplă și se num. pipeline freezing sau flushing. Situația arată astfel:

(condition, NEW PC)

```
i   branch          IF  ID  EX  MEM  W/B.
i+1 branch successor    IF  stall stall stall IF  ID  EX  MEM  W/B.
i+2 branch successor+1                    IF  ID  EX  MEM  W/B.
i+3                                           IF  ID  EX  MEM  W/B.
```

3 stall cycles.

Exemplu:
```
Test: if X>10 goto Finish
      X = X * 2
Finish: print(X);
```

Tehnica pipeline freezing introd 3 cicluri de stall, ceea ce la un pipeline pe 3 stagii și la incidență 30% de instr. de salt duce la diminuarea drastică a perf. permise de pipeline.

Reducerea pierderilor cauzate de aceste opriri implică 2 măsuri:

1. evaluarea mai devreme a condiției

→ la DLX instr. de salt sunt { branch if equal to 0 / branch if not equal to 0 } care implică testarea de 0 a unui registru. Această testare de 0 ar putea fi mutată din stagiul EX în stagiul ID. Fiind vorba despre 2 posibile val pt PC este necesar ca ambele val. să fie calculate devreme.

2. compute PC+4, NPC → EX (MEM)

Astfel încât, dacă se reușește acest lucru, sit. se modifică. (în urma acestor 2 modif. 1; 2) astfel:

```
i      IF  ID  EX  MEM  WB
i+1        IF  stall IF  ID  EX  MEM  WB
i+2                   IF  ID  EX  MEM  WB
i+3                       IF  ID  EX   MEM  WB
```

- am nevoie de un ciclu de stall și în sit. clasică merge în mod normal.

Repetarea celor 2 cicluri de fetch este necesară pt a se asigura aducerea din mem. în mod corect a unui instr. care trebuie exec. Totuși în cazurile în care branch-ul a fost

not taken, primul fetch a fost făcut în mod corect ⇒
⇒ repet. lui va duce la pierderea încă unui ciclu de clock.
Această situație va putea fi evitată prin utilizarea de mec. de predicție pt instr. de salt.

Există câteva posibilități date de compil. pt gestionarea sit. de branch. Cea mai simplă = pipeline freezing = oprirea pipeline-ului până la terminarea instr. de salt, moment în care se cunoaște certitudinea dacă pipeline-ul se ramifică sau nu și adresa de salt.

O altă alternativă cu perf. mai bună și doar puțin mai complexă = a trata fiecare branch ca fiind not taken =

= predict not taken (PNT)

Mec. PNT implică cont. exec. instr branch succesor, chiar dacă nu există certit că aceasta e cea care va trebui exec. Bineînțeles BS nu trebuie lăsat să intre în EX înainte de cunoașterea a adr. de salt.

BS - branch succesor.

i branch	IF	ID	EX	MEM	W/B	
i+1 BS		IF	ID	EX	MEM	W/B
			IF	ID	EX	MEM W/B.

FIC - curs 10

25.11.2016.

- pipeline freezing / flushing / stop.
- predict untaken
- predict taken

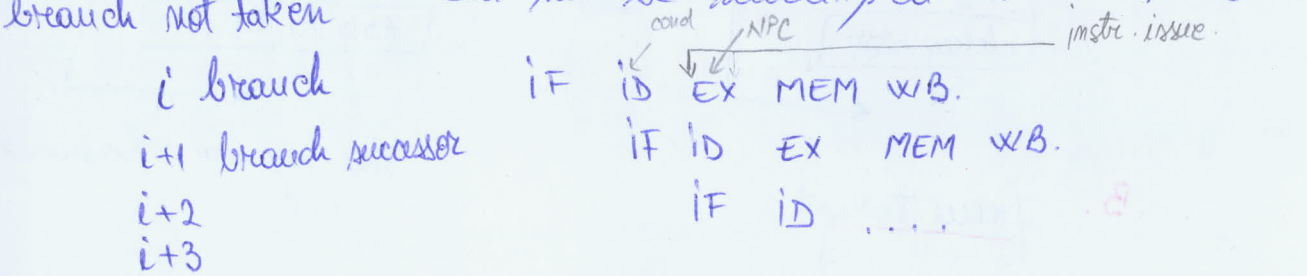
```
for ( i=0; i<100; i++)
    {
      ;
    }
```

- delayed branch.

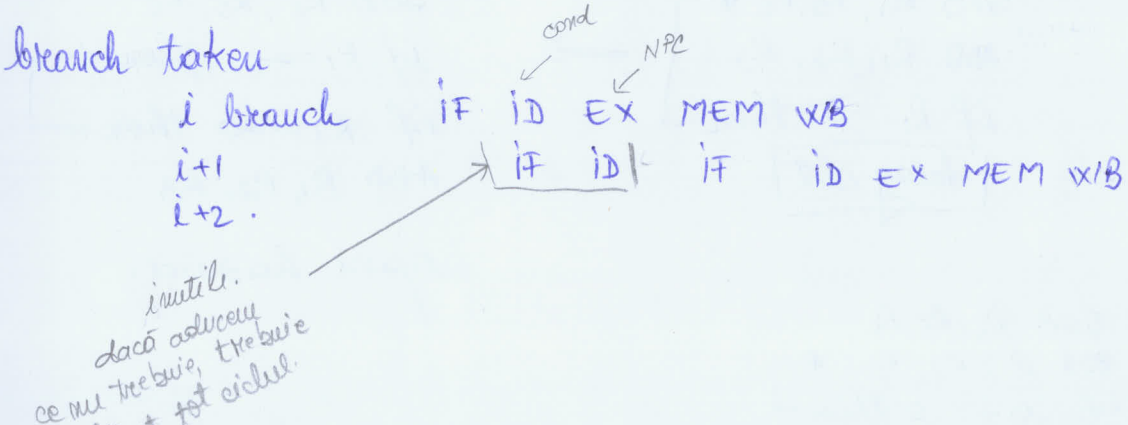
* pipeline freezing → cea mai simplă
  → presup. oprirea pipeline-ului
  → cele mai mari penalități, dar nu necesită alte măsuri

* predict untaken → asigură performanță crescută, se poate atinge doar cu o investiție minimă
  → ideea de bază: controlul execuției prog. ca și când saltul nu se execută
  → exec. în cont. e permisă atâta timp cât nu se modifică starea mașinii

branch not taken



branch taken



* predict taken → presupune saltul instrucțiunii la adresa calculată, dar în cazul DLX, calculul este ef. după eval. condiției =)
se știe cu certitudine dacă saltul se ef. sau nu. Deci, această metodă nu aduce avantaje.

*. delayed branch → se utilizează în multe unități uprograme.
→

i branch instr
i+1 br. succ.
i+2 br. succ.
⋮
i+M br. succ.
i+M+1 br. target.

} delay slots. secvențial.

Cei M succesori ocupă niște sloturi de timp = delay slots.
Nr. de delay slots depinde de arhi. și lungimea pipeline
(e unic și fiecare în parte).

DLX → 1 delay slot.

A. from before.

ADD R_1, R_2, R_3
if R_2 == 0 then
delay slot

→

if R_2 == 0 then
ADD R_1, R_2, R_3

nu apare hazard.

B. from target

SUB R_4, R_5, R_6 ←
ADD R_1, R_2, R_3
if R_1 == 0 then
delay slot

⟹

SUB R_4, R_5, R_6
,
if R_1 == 0 then ← sau
ADD R_1, R_2, R_3

apare hazard.

sau

SUB R_4, R_5, R_6 ←
ADD R_1, R_2, R_3
if R_1 == 0 then
SUB R_4, R_5, R_6

Nu st. de ex. pt.

pe când celelalte vor avea nevoie de mai multi cicli de tact. ~~Scenar~~

Scenariul DLX multiciclu arată astfel:

- EX → int
- A_1, A_2, A_3, A_p → adunare floating point
- $M_1, M_2 \ldots M_7$ → înmulțire în —"—

MULD	IF	ID	M_1	M_2	M_3	M_4	M_5	M_6	M_7	MEM	WB	
ADD D		IF	ID	A_1	A_2	A_3	A_4	MEM	WB			
LD			IF	ID	EX	MEM	WB					
SD.				IF	ID	EX	MEM	WB				

Dacă s-ar folosi o unică unitate de exec, aceste instr ar provoca hazard structural, de aceea extinderea pipeline pt. DLX multiciclu. ?

care să trateze operațiile aritm. diferite. De aceea comb. indicată pre2. situația unui hazard structural. Lungimea dif. a stp. de exec. modifică situațiile de hazard din pipeline.

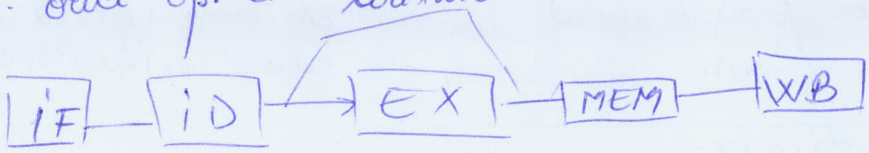
c. from fall-through

```
ADD R1, R2, R3                    ADD R1,R2,R3
if R1==0 then                     if R1==0 then
  delay slot                        OR R7, R8, R9
  OR R7, R8, R9          →          SUB R4, R5, R6
  SUB R4,R5,R6
```

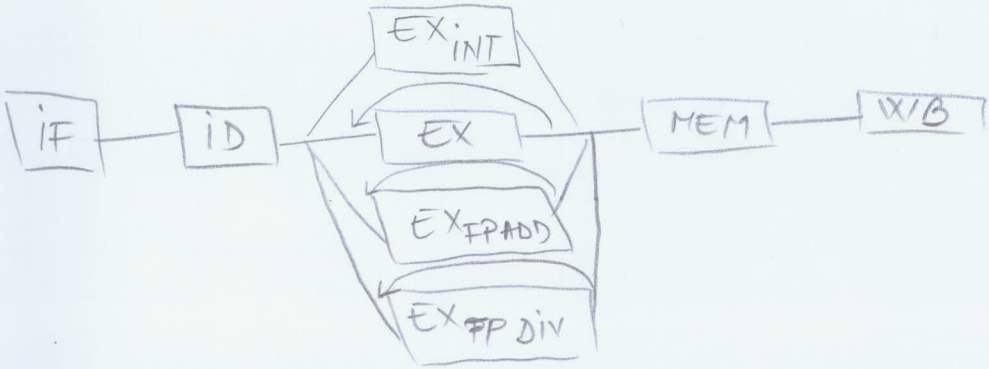
delayed branch ← Această tehnică prez. inconv. d-ce nu tot timpul compilatorul reuseste să găsească instr. care să nu modifice logica programului, fiind posibile situații de hazard.

3.4. Extinderea DLX pt. operații multiciclu.

Până acum am considerat că ALU ef. operații cu întreg. Dacă ALU poate ef. și op. în virgulă flotantă, devine nepractic scenariul în care stag. de exec. durează 1 singur tact pt. orice op. ~~alt~~ aritmetică.



O posib. sol. este prelung. ciclului de tact. Altă sol, cea pref, presup ? execuției în lățimii de-a lungul mai multor cicluri de tact. Pipeline-ul pe 5 stagii se modif. a.i. va exist. mai stagii EX pt întreg, unul pt. FP add, unul pt FP multiply și unul pt FP div.



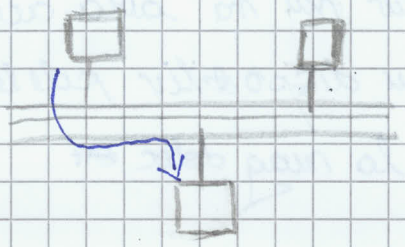
FIC - S14. - 13.01.2017.

1. IAS, set de instrucțiuni
2. Microprog.
3. Pipeline
4. Magistrale și periferice

Magistrala -

rol: interconectarea procesorului la memorie și periferice

Magistrala
- pros: pret, versatilitate
- cons: viteza / latime de banda
  - consum de energie



O operație pe magistrală implică un mecanism de cadrare și o operație de transfer de date

D-ce există m. multe dispozitive care partajează o mag. doar unul inițializează preluarea mag. în vederea transferului de date

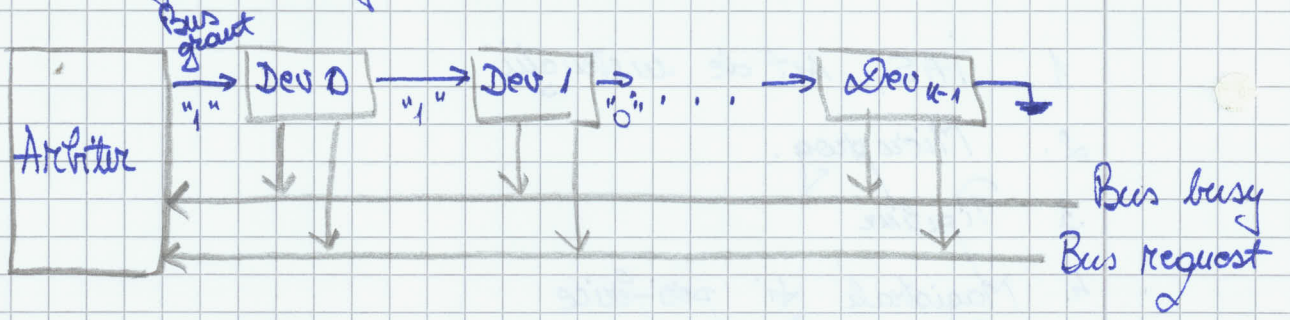
Master → al care controlează.

Slave

D-ce pot exista m. mulți masteri simultan este necesară arbitrarea. Arb. poate fi făcută cu ajutorul

- daisy chaining
- polling
- independent requesting

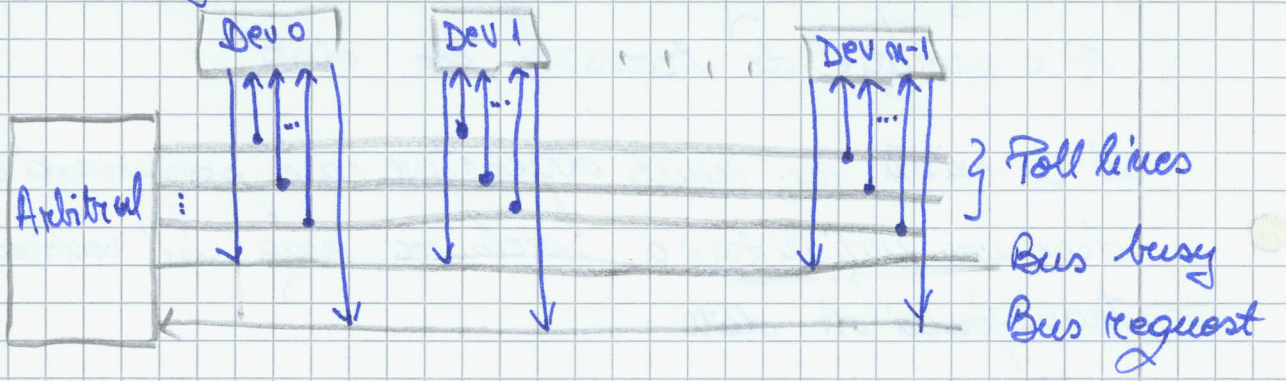
A. Daisy chaining



Disp care a cerut magistrala nu lasă să treacă mai departe
În plus, ~~se~~ setează și pe bus busy.
Are prioritate cel care e mai aproape de arbitru.

Avantaje: - cea mai simplă metodă
Dezavantaje: - ultimul dispozitiv nu ra avea acces la mag.
- dacă se defectează un dispozitiv celelalte ~~se~~
nu vor avea acces la mag delec ~~ei~~

B. Polling



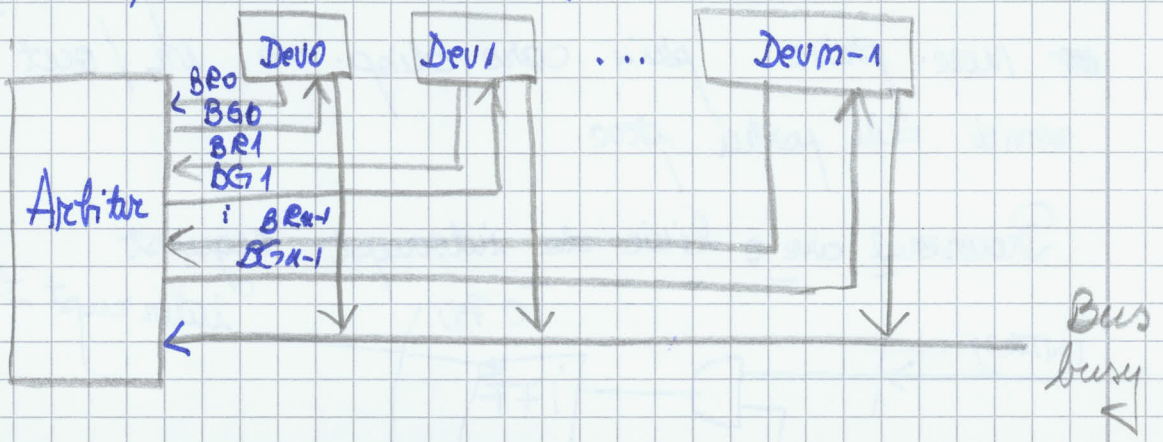
În mod ciclic arbitrul generează valori pe liniile de poll pt a „întreba" dispozitivele dacă solicită magistrala.
În momn în care val de pe liniile de pol. repr. adresa unui disp acesta are dreptul de a prelua magistrala

+: arbitrul selectează într-un mod. echitabil perifericele care apoi devin master pe mag.

–: nr liniilor

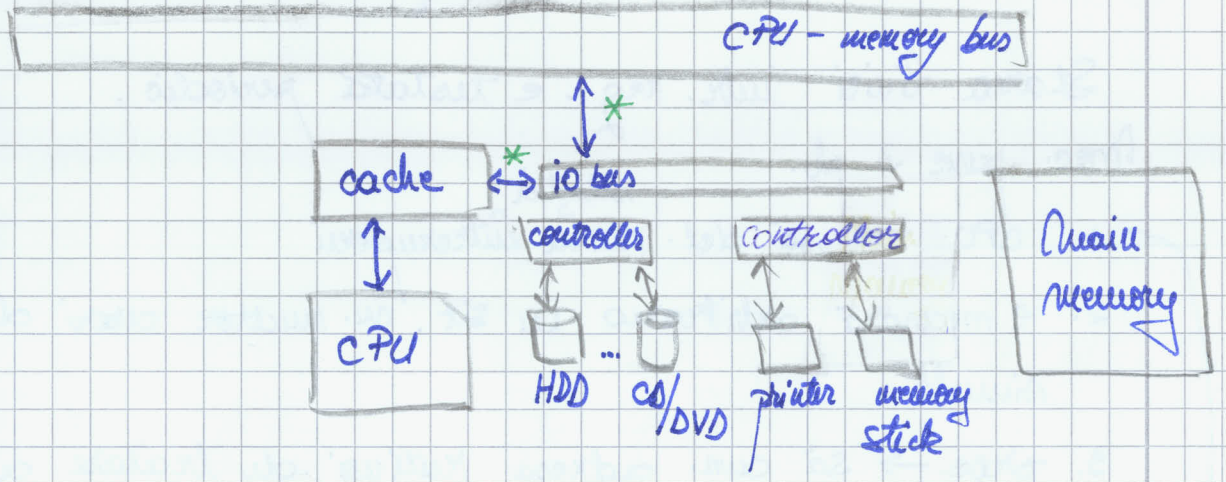
arb. interoghează tot, nu doar pe cele care au nevoie.

c. Întâ dependent requesting



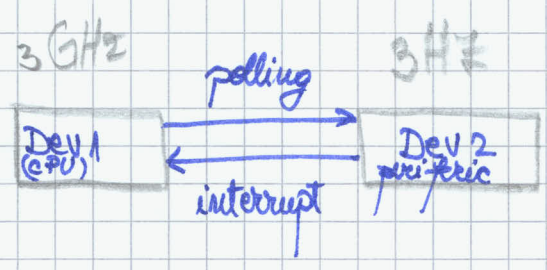
+: → pa mai eficientă soluție

–: → pa mai costisitoare



* north bridge.

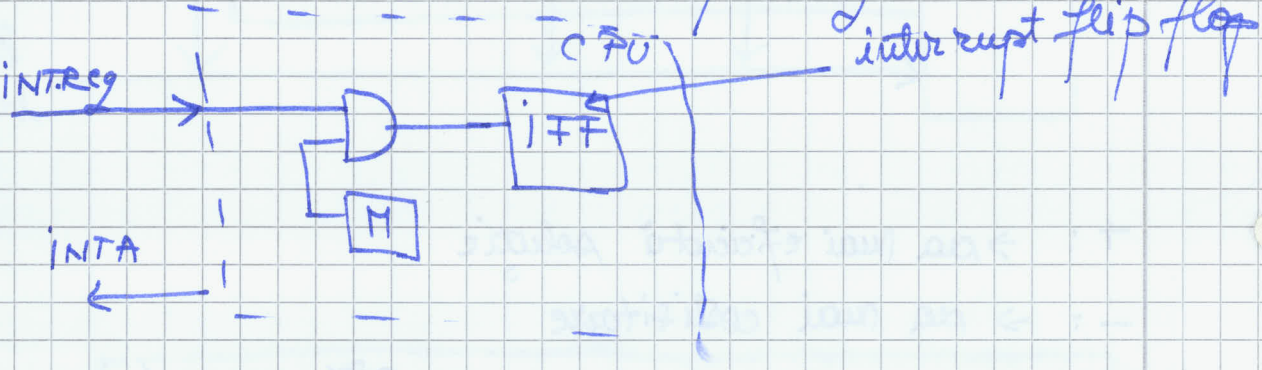
IDE, E-IDE, S-ATA



Polling merge doar dacă dif de viteză nu e mare, sau Dev.2 e activă (când îl întrebi să răspundă)

Mec. de polling → nu e potrivit pt sit. în care perif. slave e m. m. lent decât cel master, fiind deci necesar un mec. prin care perif. își anunță terminarea trans. Acest mec = întrerupere și este mec. prin care disp. de in/out solicită servicii din partea proc.

Procesorul are o linie de interrupt request

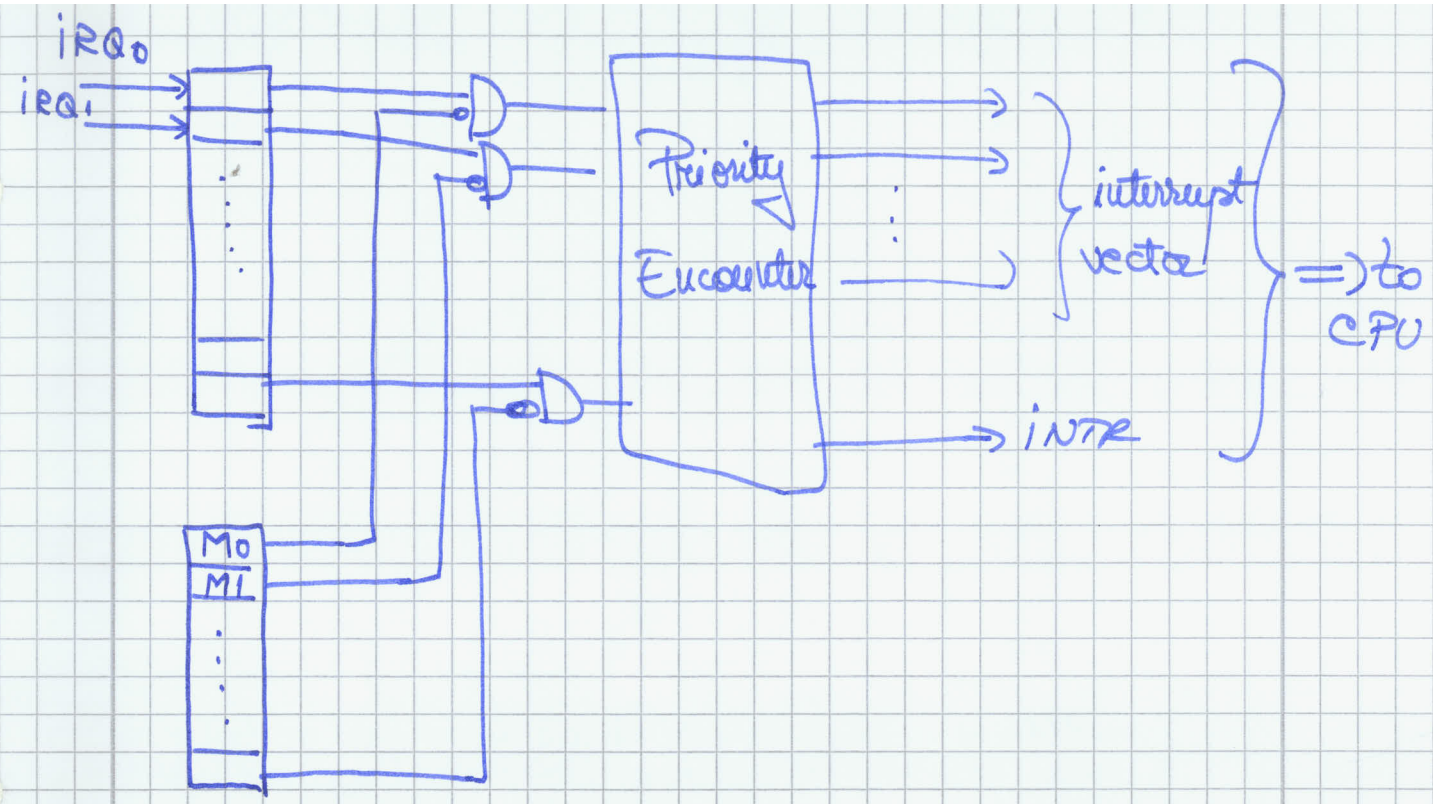


Starea liniei intr. req. e testată periodic.

Mec. pune 3 pb.

1. CPU → să det. sursa întreruperii (selecția)
2. e necesară arbitrarea în sit. m. multor cereri de intr. simult
3. proc → să cun. adresa rutinei de tratare a intr.

→ Single Line Selection
→ Multiple Line Selection
→ Vector interrupts


IRQ0
IRQ1
Priority
Encoder
interrupt
vector
=> to
CPU
INTR
M0
M1